

ChaletsBooking (IARS)

Hybrid Personalized Recommendation Engine

Final Year Project - Complete Technical Documentation

Project Title: Intelligent Accommodation Recommendation System (IARS) / ChaletsBooking

Techn

Table of Contents

1. [Introduction](#1-introduction)

2. [Pro

1. Introduction

ChaletsBooking (IARS - Intelligent Accommodation Recommendation System) is a web-based chalet booking platform developed as a Final Year Project. The system allows property owners to list chalets, administrators to approve listings, and guests to search, book, pay (demo), and review properties.

A core AI component of this project is the Hybrid Personalized Recommendation Engine, which extends a baseline Bayesian Ranking + Sentiment Analysis recommendation system with user_id-based personalization. Unlike a generic "best properties for everyone" list, the engine produces different ranked recommendations for each guest based on their individual search history, property views, booking history, and review ratings-while ensuring that poor-quality properties (low Bayesian score or negative sentiment) cannot rank highly regardless of preference match.

This document describes the complete design, implementation, algorithms, database schema, API endpoints, and frontend integration of the recommendation system.

2. Problem Statement

2.1 Current State (Before Personalization)

Initially, ChaletsBooking ranked all chalets the same way for every guest using aggregate review quality and sentiment scores. Every guest saw the same "best" chalets in the same order, regardless of who they were or what they had looked at before.

2.2 The Gap

The system had no concept of an individual guest's preferences or history. There was no user_id linking a guest to their past searches, bookings, or reviews. "Best chalet" was defined only as "objectively best by aggregate rating"-not "best for this guest."

2.3 Solution

A new Hybrid Personalized Recommendation Engine was implemented that:

1. Tracks all authenticated guest interactions using **user_id**

2. Bui

3. Project Objectives

| Objective | Status

1 | Imp

4. System Architecture

4.1 High-Level Architecture

```

????????????????????????????????????????????????????????????
?                               React Frontend (Port 5173)          ?
?  HomePage ? SearchPage ? PropertyDetailPage ? RecommendedProps ?
????????????????????????????????????????????????????????????
?                               ? HTTP / REST (Axios + JWT)        ?
????????????????????????????????????????????????????????????
?                               Node.js + Express (Port 5000)      ?
?  searchController ? propertyController ? reviewController      ?
?  personalizedRecommendationEngine ? buildUserProfile            ?
?  bayesianRanking ? collaborativeFiltering ? propertySimilarity ?
????????????????????????????????????????????????????????????
?                               ? Mongoose                        ?
?                               ? Axios                          ?
????????????????????????????????????????????????????????????
?  MongoDB (iars_db)      ? ? Python FastAPI Sentiment Service ?
?  Users, Properties,    ? ? TF-IDF + Logistic Regression      ?
?  Bookings, Reviews,    ? ? Port 8000                          ?
?  SearchHistory,        ? ??????????????????????????????????
?  PropertyValue         ?                                     ?
????????????????????????????????????????????????????????????

```

4.2 Recommendation Data Flow

```

Guest Activity (linked to user_id)
?
??? SearchHistory      (saved on GET /api/search)
??? PropertyValue      (saved on POST /api/properties/:id/view)
??? Booking            (existing booking module)
??? Review             (existing review module + sentiment)
?
?
buildUserRecommendationProfile(userId)
?
?
scoreAllEligibleProperties()
  ??? computeBayesianQualityScore()
  ??? computeSentimentQualityScore()
  ??? computeGuestRelevanceScore()
  ?    ??? Content similarity (42%)
  ?    ??? Collaborative filtering (28%)
  ?    ??? Preference alignment (30%)
  ??? finalScore = quality x (0.40 + relevance x 0.60)
  ?
  ?
buildStructuredRecommendations() OR buildColdStartRecommendations()
?
?
GET /api/recommendations -> RecommendedProperties.jsx

```

5. Technology Stack

Layer | Technology | Purpose

Fronte

6. Database Design

6.1 SearchHistory Collection

Stores each authenticated guest's search queries.

Field | Type | Description

user |

Rules: Maximum 20 records per user; oldest entries deleted when exceeded.

File: server/models/SearchHistory.js

6.2 PropertyView Collection

Stores property view/click events for authenticated guests.

Field | Type | Description

user |

Rules: Unique index on { user, property }; maximum 50 views per user.

File: server/models/PropertyView.js

6.3 Booking Collection

Field | Type | Description

guest |

File: server/models/Booking.js

6.4 Review Collection

Field | Type | Description

guest |

File: server/models/Review.js

6.5 Property Collection (Sentiment Snapshot)

Key fields used by recommendation engine:

Field | Type | Description

title, de

File: server/models/Property.js

7. User Activity Tracking (user_id-Based)

Personalization requires linking every interaction to a user_id. This was the prerequisite blocker identified in the FYP problem statement.

7.1 Search History Tracking

When: Guest performs a property search while logged in.

Where

7.2 Property View Tracking

When: Authenticated guest opens a property detail page.

Where

7.3 Booking History

Source: Existing Booking model where guest field references User._id.

Used

7.4 Review / Rating History

Source: Existing Review model where guest field references User._id.

Used

7.5 Recency Weighting

Only the latest 5 searches, bookings, and views are used. Recency decay weights:

```
[1.0, 0.72, 0.52, 0.38, 0.28]
```

Recent activity has higher influence than older activity.

8. User Profile Building

Module: server/utls/buildUserProfile.js

Funct

8.1 Profile Fields Generated

Category | Fields

Identit

8.2 Interaction Weights

Source | Weight

Bookin

8.3 Property Type Inference

Inferred from title/description keywords: chalet, villa, apartment, resort.

8.4 Amenity Extraction

Keywords extracted from descriptions: wifi, parking, pool, breakfast, kitchen, ac, heater, balcony, garden, etc.

9. Recommendation Engine Overview

Main module: server/utls/personalizedRecommendationEngine.js

The engine extends (does not replace) the existing Bayesian + Sentiment system.

9.1 Key Constants

Constant | Value | Purpose

RECO

9.2 Main Functions

Function | Purpose

scoreA

9.3 Engine Modes

Mode | Engine String | Layout

Person

10. Bayesian Ranking Algorithm

Module: server/utils/bayesianRanking.js

Bayesian averaging prevents properties with few reviews from appearing artificially high or low.

10.1 Bayesian Rating Formula

```
bayesianRating = (avgRating x reviewCount + PRIOR_MEAN x PRIOR_WEIGHT)
                  / (reviewCount + PRIOR_WEIGHT)
```

PRIOR_MEAN = 3.5

PRIOR_WEIGHT = 4

10.2 Quality Score (with reviews)

```
quality = (bayesianRating/5) x 0.30
          + (positivePercent/100) x 0.40
          + logNorm(reviewCount) x 0.15
          + availabilityScore x 0.08
          + amenityScore x 0.05
          - negativeShare x 0.20
```

10.3 Penalties

- High star rating (>=4) but positive sentiment < 50%: -0.12

- Baye

10.4 No Reviews

Base score: $0.12 + \text{availability} \times 0.10 + \text{amenity} \times 0.06$

10.5 Role in Recommendations

Bayesian quality acts as a trust floor. Properties below MIN_QUALITY_SCORE (0.28) receive recommendationScore = 0 in personalized mode and are excluded from recommendations.

11. Sentiment Analysis Integration

11.1 Python Sentiment Service

Location: sentiment-service/

Mode

Response includes:

- sent

11.2 Node.js Integration

Client: server/services/sentimentService.js

Env v

Flow on review submission:

1. Gu

11.3 Sentiment Quality Score

```
sentimentQuality = positivePercent/100 - (negativeCount/reviewCount) x 0.35
```

Combined with Bayesian:

combin

12. Content-Based Filtering

Module: server/utls/propertySimilarity.js

Answers: `"Which chalets are most similar to what this guest has viewed, booked, or rated highly?"`

12.1 Content Similarity Formula

```

similarity = cityMatch x 0.32
            + priceSimilarity x 0.28
            + amenityJaccard x 0.22
            + sentimentSimilarity x 0.18

```

12.2 Price Similarity Tiers

Price difference | Score

<= 129

12.3 Weighted Content Score

Interactions are weighted by recency and source (booking > review > view):

```
contentRelevance = weighted average of similarity to each interaction
```

Weight in guest relevance: 42%

13. Collaborative Filtering

Module: server/utils/collaborativeFiltering.js

Answers: `""Which chalets are most liked by guests similar to this one?"`

13.1 Co-Occurrence Matrix

Built from:

- Bool

For each guest, all pairs of properties they interacted with increment matrix scores.

13.2 Collaborative Score

For each property user interacted with:

```

score += min(matrix[sourceId][candidateId] / 4, 1)
collaborativeScore = average(score)

```

Weight in guest relevance: 28%

14. Hybrid Scoring Formula

14.1 Guest Relevance Score

```

guestRelevance = contentRelevance x 0.42
                + collaborativeRelevance x 0.28
                + preferenceRelevance x 0.30

```

Preference relevance includes: latest booking city, recent search cities, preferred city, budget match, view similarity, booking history, review favourites, search filter match, property type, amenities, availability.

14.2 Final Recommendation Score

Personalized (with history):

Trust floor:

Cold start (no history):

14.3 Why Multiplicative (Not Additive)

Personalization never overrides quality. A chalet matching the guest's budget but with terrible sentiment receives score 0-not a high rank. This satisfies the FYP constraint that Bayesian and sentiment remain quality constraints.

14.4 Section Composite Score

Within each section, properties are ranked by:

```
sectionScore = recommendationScore x 0.48
              + sectionMatchScore x 0.28
              + combinedQuality x 0.14
              + ratingSentimentBoost x 0.10
```

15. Personalized Recommendation Sections

When a guest has interaction history, up to 6 sections are shown (max 4 properties each, max 24 total, no duplicates).

| Section ID | Title | Selection Logic

Sections with no matching data are skipped (e.g., no views -> Section 4 hidden).

16. Cold Start Strategy

Trigger: Guest not logged in OR authenticated user with zero interaction history.

16.1 Cold Start Sections

| Section ID | Title | Ranking

16.2 Why Cold Start Matters

New users must still receive useful recommendations. The existing Bayesian + Sentiment engine serves as a legitimate fallback-a strength compared to systems with no defined cold-start strategy.

17. Recommendation Rules and Quality Constraints

Rule | Implementation

No dup

18. Match Reasons (Explainability)

Each recommended property displays up to 3 reason badges for FYP explainability.

Reason Key | Display (English)

similar

Backend: buildHybridMatchReasons() in engine + enrichMatchReasons() in enrichment module.

Fronte

19. API Specification

19.1 GET /api/recommendations

Auth: optionalProtect (works for guests and anonymous users)

Contr

Query Parameters:

Parameter | Type | Description

city | s

Response:

```
{
  "success": true,
  "count": 20,
  "engine": "hybrid-personalized-sectioned",
  "layout": "personalized",
  "personalized": true,
  "profileSignals": {
    "userId": "6a3efa7587fec180a9b3e297",
    "trackingEnabled": true,
    "mode": "hybrid_personalized",
    "hasSearchHistory": true,
    "hasBookings": true,
    "hasViews": true,
    "preferredCity": "Lahore",
    "preferredPrice": 15000,
    "relevanceSignals": {
      "contentBased": true,
      "collaborativeFiltering": true,
      "qualityFloor": "bayesian_sentiment"
    }
  },
  "sections": [
    {
      "id": "recent_searches",
      "titleKey": "search.recommendationSection.recentSearches",
      "items": [ { "_id", "title", "recommendationScore", "matchReasons", ... } ]
    }
  ],
  "data": [ ... ]
}
```

19.2 GET /api/search

Saves SearchHistory for authenticated users.

19.3 POST /api/properties/:id/view

Records PropertyView for authenticated guests.

20. Frontend Implementation

20.1 RecommendedProperties.jsx

Location: client/src/components/search/RecommendedProperties.jsx

- Fetches `GET /api/recommendations` with search context params

- Re-f

20.2 HomePage.jsx

- Top: search form + 12 available properties preview

- Bott

20.3 SearchPage.jsx

- Full search results + ``<RecommendedProperties />`` below

20.4 PropertyDetailPage.jsx

- Calls ``trackPropertyView(id)`` when authenticated guest views property

- Feed

20.5 usePropertySearch.js

Provides recommendationContext:

{ city,

21. File Structure and Module Reference

```
server/
??? models/
?   ??? SearchHistory.js      # Guest search tracking
?   ??? PropertyView.js      # Guest view/click tracking
?   ??? Booking.js           # Booking history
?   ??? Review.js            # Ratings + sentiment
?   ??? Property.js           # Listings + sentimentSnapshot
??? controllers/
?   ??? searchController.js   # /search, /recommendations
?   ??? propertyController.js # trackPropertyView
??? utils/
?   ??? personalizedRecommendationEngine.js # MAIN ENGINE
?   ??? buildUserProfile.js     # Guest profile
?   ??? bayesianRanking.js      # Bayesian quality
?   ??? collaborativeFiltering.js # CF matrix
?   ??? propertySimilarity.js   # Content similarity
?   ??? recommendationEnrichment.js # Sentiment + reasons
?   ??? propertySignals.js      # Availability, amenities
?   ??? propertySentimentStore.js # Snapshot sync
??? routes/
?   ??? searchRoutes.js        # GET /recommendations
??? scripts/
    ??? seedProperties.js       # Demo properties
    ??? seedDemoReviews.js     # Demo reviews + bookings

client/src/
??? components/search/
?   ??? RecommendedProperties.jsx
?   ??? RecommendedPropertyCard.jsx
??? pages/
?   ??? HomePage.jsx
?   ??? SearchPage.jsx
?   ??? PropertyDetailPage.jsx
??? hooks/
```

```
?   ??? usePropertySearch.js
??? services/
    ??? searchService.js      # getRecommendations()
```

22. Setup and Installation

22.1 Prerequisites

- Node.js 18+

- Mon

22.2 Environment Variables

server/.env:

MONGO_
JWT_SE
SENTIN

client/.env:

VITE_

22.3 Install and Run

```
npm run install:all
npm run seed
npm run seed:demo:full
npm run dev
```

This starts:

- Clien

23. Demo Data and Test Accounts

23.1 Seed Commands

Command | Description

npm ru

23.2 Test Accounts

Role | Email | Password

Admin

24. Testing and Verification

Automated checks performed on the recommendation engine:

Test | Expected Result

Anony

25. Viva Demonstration Guide

Step 1: Show Cold Start

1. Open home page without logging in

2. Sho

Step 2: Show User Tracking

1. Log in as guest@iars.com

2. Sea

Step 3: Show Personalized Recommendations

1. Refresh home page

2. Sho

Step 4: Show Different Users Get Different Lists

1. Log in as guest.demo2@iars.com

2. Sea

Step 5: Explain Quality Floor

1. Point out that all recommended properties have positive sentiment

2. Exp

Step 6: Show API Response

1. Open browser DevTools -> Network -> `/api/recommendations`

26. Future Enhancements

Enhancement | Description

A/B tes

27. Conclusion

The Hybrid Personalized Recommendation Engine successfully extends ChaletsBooking's existing Bayesian Ranking and Sentiment Analysis system with user_id-based personalization. The system:

1. **Tracks** guest searches, views, bookings, and reviews

2. **B**

This implementation directly addresses the FYP problem statement and provides a measurable, demonstrable AI-based recommendation component suitable for viva evaluation.

28. References

1. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook*. Springer.

2. Res

End of Document

ChaletsBooking (IARS) - Hybrid Personalized Recommendation Engine

*FYP